Parallel and Distributed Programming
CUDA

Kenjiro Taura

2026/10/26

Contents

Overview 2
CUDA Basics 5
Kernels 7
Threads and thread blocks 16
Data sharing 20
More on choosing a block size 36

Overview

Goal

- learn CUDA, the basic API for programming NVIDIA GPUs
- where CUDA/GPU are similar to OpenMP/CPU, and where they are different?
- reference (official documentation)
  - CUDA Programming Guide

Compiling/running CUDA programs with NVCC

- compile with `nvcc` command

```
$ nvcc program.cu
```

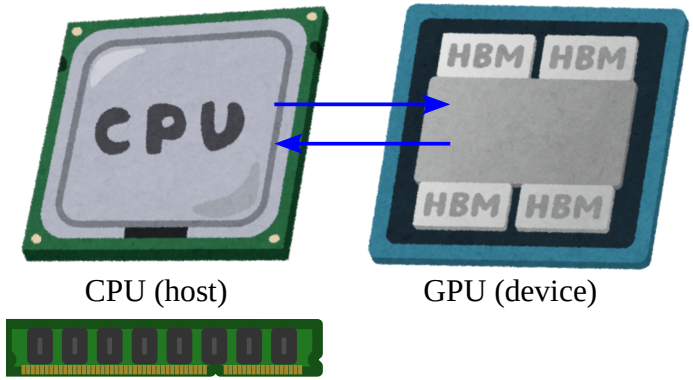
- the conventional extension of CUDA programs is `.cu`
- `nvcc` can handle ordinary C/C++ programs too (`.cc`, `.cpp` → C++)
- you can have a file with any extension and insist it is a CUDA program (convenient when you maintain a single file that compiles both on CPU and GPU)

```
$ nvcc -x cu program.cc
```

CUDA Basics

GPU is a separate device

- GPU is a device separate from CPU
  1. *code* that runs on GPU must be so declared
  2. *data* had to be copied between CPU and GPU in the past
     - it's changing now
- a GPU is often called a *"device"*,
- and a CPU a *"host"*



look at an image

Kernels

Two things to learn first: writing and launching kernels

- code that runs on GPU is called "*GPU kernel*" (or simply a "kernel")

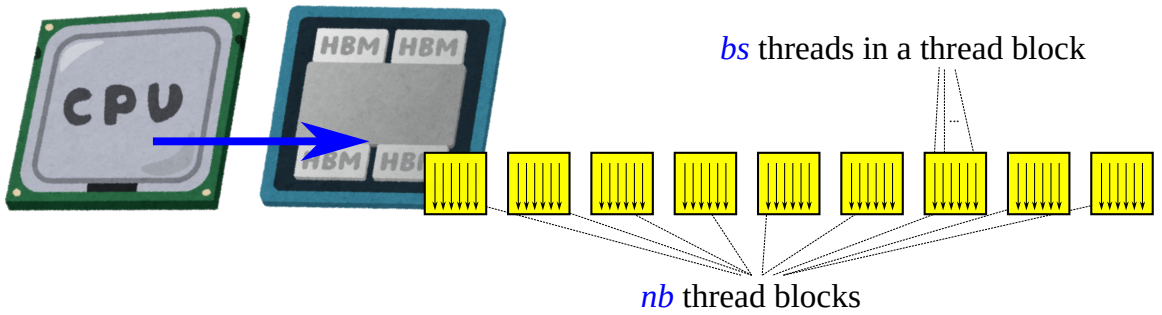
1. entry point to GPU is a C++ function defined with `__global__` keyword

   ```
   __global__ void f(...) { ... }
   ```

2. a host can launch such a function specifying the number of threads.

   ```
   f<<<nb,bs>>>(...);
   ```

   will create ($nb \times bs$) CUDA threads, each executing `f(...)`



A kernel call and the host overlap but two kernel calls do not

- when you call a kernel, the host continues execution without waiting for it to finish
- two kernel calls are serialized on the GPU side, by default
- don't forget to insert `cudaDeviceSynchronize()` when you need to wait for the kernel to finish

```
h0();
g0<<<...,...>>>();
h1();
g1<<<...,...>>>();
h2();
g2<<<...,...>>>();
cudaDeviceSynchronize();
h3();
```

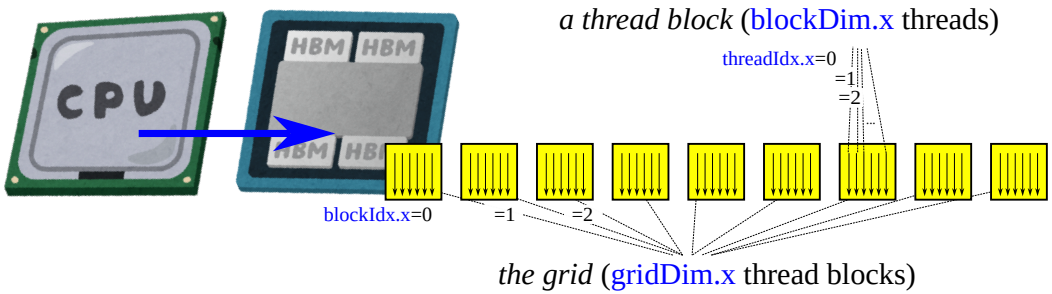
- `g0` may overlap with `h1` and `h2`
- `g0` and `g1` do not overlap because the GPU serializes them by default
- `h3` does not overlap with anything because of `cudaDeviceSynchronize()`

Thread IDs

- you get the number of threads and your ID (analogous to `omp_get_num_threads()` and `omp_get_thread_num()` in OpenMP) via `gridDim`, `block{Dim,Idx}` and `threadIdx`
- when you launch a kernel by

```
f<<<nb,bs>>>(...);
```

  - `gridDim.x` = *nb* (the number of blocks = the "grid" size)
  - `blockIdx.x` = the thread's block ID ($\in [0, \mathrm{nb})$)
  - `blockDim.x` = *bs* (the thread block size)
  - `threadIdx.x` = the thread ID within the block ($\in [0, \mathrm{bs})$)



Thread IDs

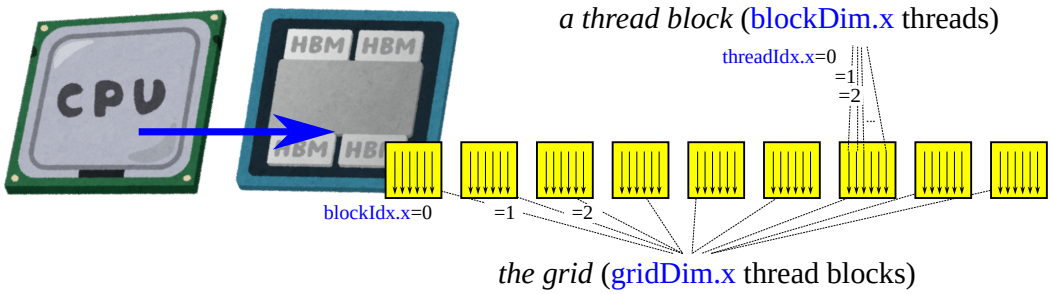
therefore a kernel can obtain:

- the total number of threads executing the kernel by:

```
gridDim.x * blockDim.x
```

- your rank by :

```
blockDim.x * blockIdx.x + threadIdx.x
```



A template for parallelizing a (1D) for loop

- given a for loop

```
for (i = 0; i < n; i++)
  S(i)
```

1. write a kernel executing a single iteration:

```
__global__ void S_fun(int n) {
  int i         = blockDim.x * blockIdx.x + threadIdx.x;
  if (i < n)
    S(i)
}
```

2. launch n threads (perhaps more than n depending on thread block size)

```
int bs = 64; // more on this later
int nb = (n + bs - 1) / bs; // ensure nb * bs >= n
S_fun<<<nb,bs>>>(n);
```

A CUDA thread vs. OpenMP thread : major difference

- launching 100000 CUDA threads (≫ the number of SMs) or making a CUDA threads for *one* iteration is quite common in CUDA (GPU)

```
f<<<1024,256>>>(...);
```

- ***each SM is capable of multiplexing / interleaving many threads simultaneously***
- on CPU, launching threads ≫ the number of cores is almost always a bad idea
- CPU-way of doing this is to partition iterations among a modest (the actual core count) number of threads

```
#pragma omp parallel for
for (i = 0; i < 1024 * 256; i++) { f(); }
```

```
OMP_NUM_THREADS=<a modest number> ./a.out
```

- ***it's not that GPU parallelism is three orders of magnitude larger than CPU***
  - CPU has parallelism *inside* each core we haven't untapped (SIMD and perhaps ILP)

nb and *bs* can be 2/3-Dimensional

- as hinted by `.x`, grid/block sizes can be multidimensional
- they are `dim3` type, a `struct` having `.x`, `.y`, `.z`
- the previous code assumes they are 1D

1D:

```
int nb = 100;
int bs = 256;
// 100*256 threads
f<<<nb,bs>>>(...);
```

2D:

```
dim3 nb(10,10);
dim3 bs(8,32);
// 10*10*8*32 threads
f<<<nb,bs>>>(...);
```

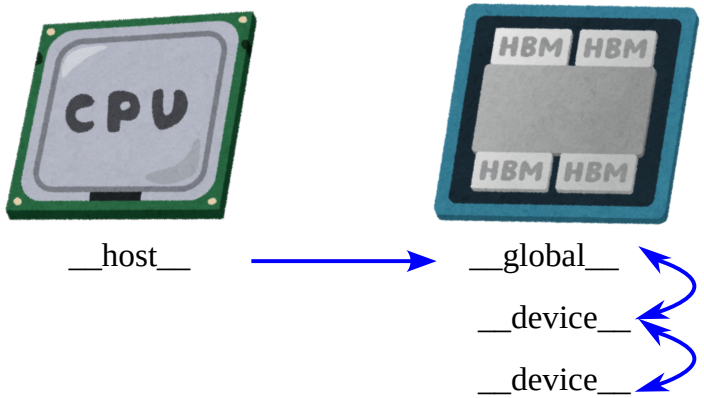
3D:

```
dim3 nb(10,5,2);
dim3 bs(8,8,4);
// 10*5*2*8*8*4 threads
f<<<nb,bs>>>(...);
```

Keywords for functions

- three keywords for functions: `__global__`, `__device__`, `__host__`

	code runs on	callable from
`__global__`	device	host/device
`__device__`	device	device
`__host__`	host	host



- `__global__` functions must be `void` (cannot return a value)
- without any keyword, it is `__host__`
- you can have *both* `__host__` and `__device__` for a function, which generates two versions (the only case `__host__` is necessary)

Macros

- convenient when writing a single file that works both on CPU and GPU
- `__NVCC__` : a macro defined when compiled by nvcc

```
#ifdef __NVCC__
  // GPU implementation
#else
  // CPU implementation
#endif
```

- `__CUDA_ARCH__` : a macro defined when compiled for device

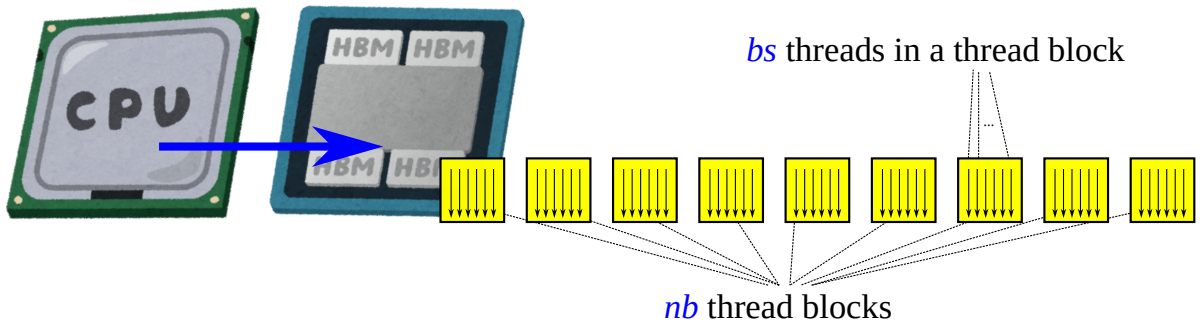
```
__device__ __host__ f(...) {
#ifdef __CUDA_ARCH__
  // device code
#else
  // host code
#endif
}
```

Threads and thread blocks

Threads and thread blocks (recap)

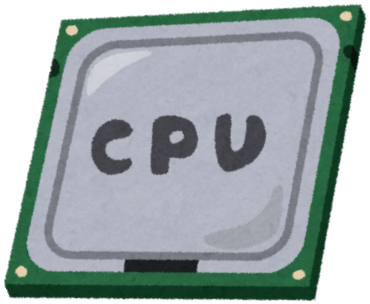
- a kernel specifies the action of *a single CUDA thread*
- when you launch a kernel you specify
  - the number of thread blocks (*nb*) and
  - the thread block size = the number of threads in a single thread block (*bs*),

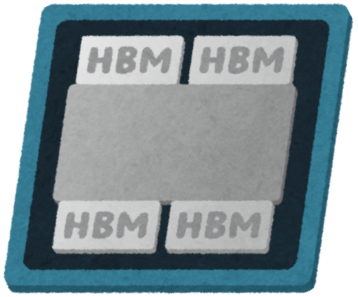
  to effectively create ($nb \times bs$) threads



but why do you need two separate numbers?

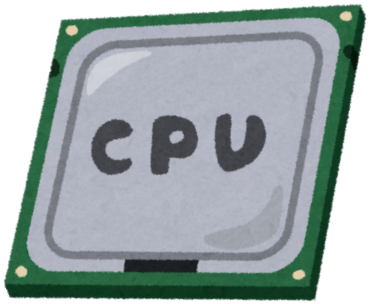
Execution model

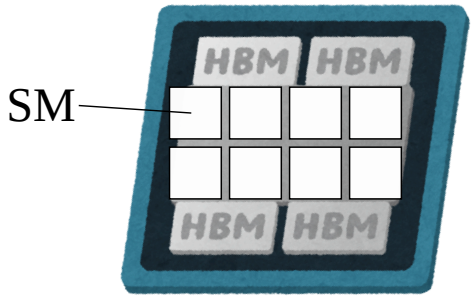




Execution model

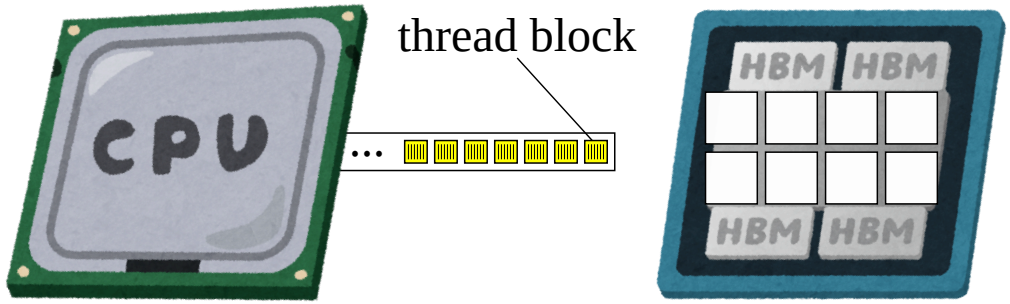
- a GPU device has a number of (132 in GH200) ***Streaming Multiprocessors (SMs)***





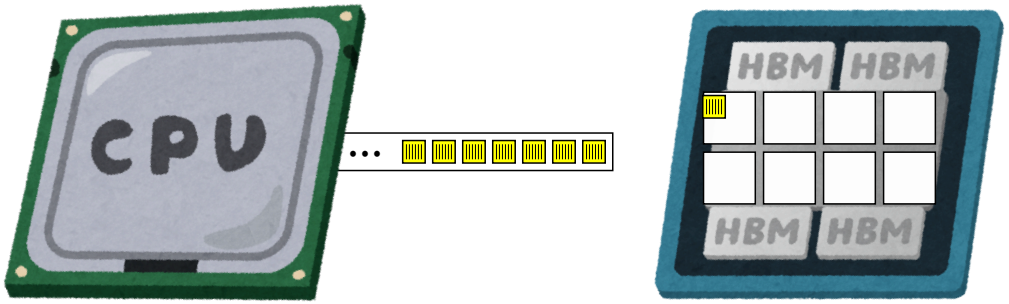
Execution model

- a GPU device has a number of (132 in GH200) ***Streaming Multiprocessors (SMs)***
- launched thread blocks line up in a queue (***stream***)



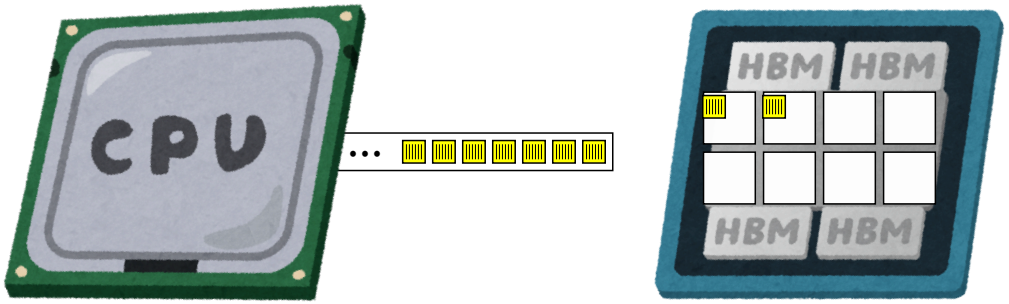
Execution model

- a GPU device has a number of (132 in GH200) ***Streaming Multiprocessors (SMs)***
- launched thread blocks line up in a queue (***stream***)
- ***a single thread block is dispatched to a single SM as a unit***



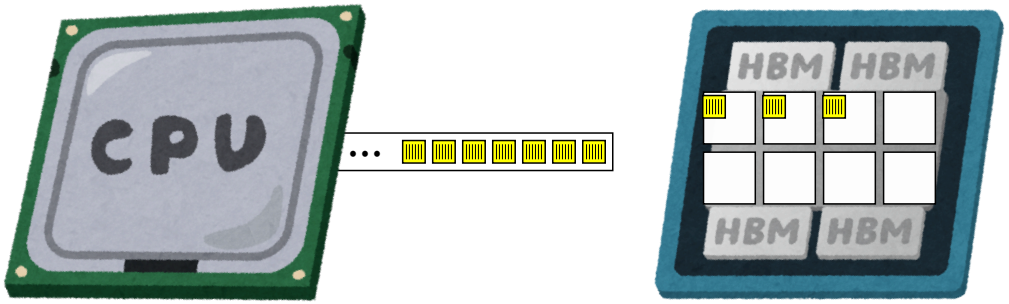
Execution model

- a GPU device has a number of (132 in GH200) ***Streaming Multiprocessors (SMs)***
- launched thread blocks line up in a queue (***stream***)
- ***a single thread block is dispatched to a single SM as a unit***



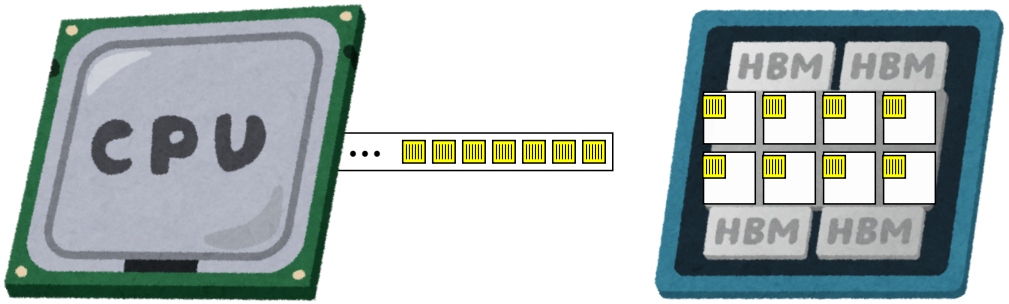
Execution model

- a GPU device has a number of (132 in GH200) ***Streaming Multiprocessors (SMs)***
- launched thread blocks line up in a queue (***stream***)
- ***a single thread block is dispatched to a single SM as a unit***



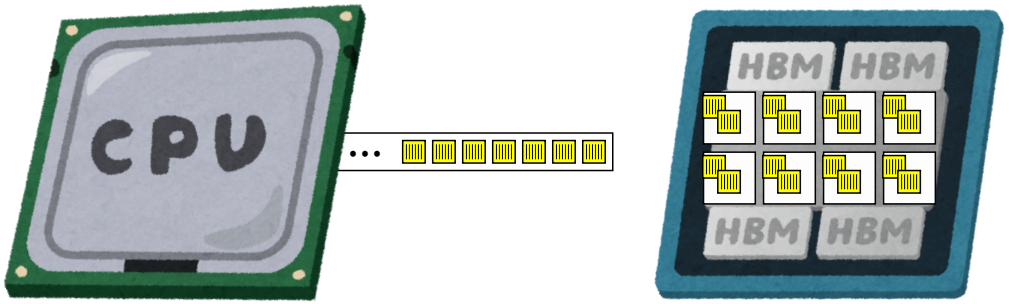
Execution model

- a GPU device has a number of (132 in GH200) ***Streaming Multiprocessors (SMs)***
- launched thread blocks line up in a queue (***stream***)
- ***a single thread block is dispatched to a single SM as a unit***



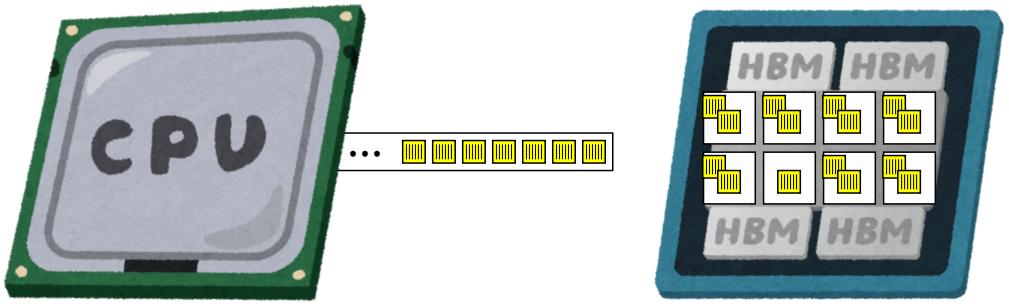
Execution model

- a GPU device has a number of (132 in GH200) ***Streaming Multiprocessors (SMs)***
- launched thread blocks line up in a queue (***stream***)
- ***a single thread block is dispatched to a single SM as a unit***
- multiple blocks may run on a single SM simultaneously, up to the *resource limit* of an SM (more on this later)



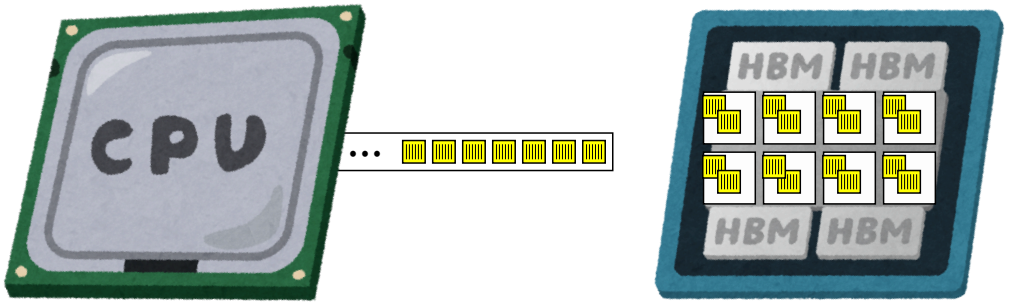
Execution model

- a GPU device has a number of (132 in GH200) ***Streaming Multiprocessors (SMs)***
- launched thread blocks line up in a queue (***stream***)
- ***a single thread block is dispatched to a single SM as a unit***
- multiple blocks may run on a single SM simultaneously, up to the *resource limit* of an SM (more on this later)
- GPU keeps SMs occupied with the *right* number of blocks

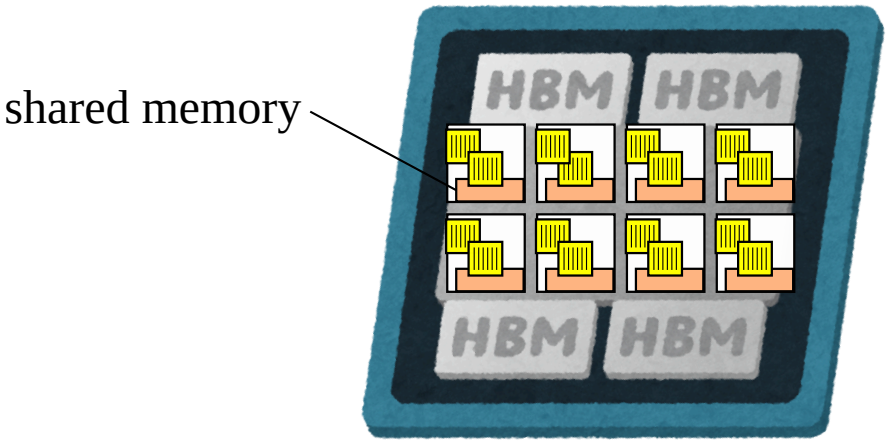


Execution model

- a GPU device has a number of (132 in GH200) ***Streaming Multiprocessors (SMs)***
- launched thread blocks line up in a queue (***stream***)
- ***a single thread block is dispatched to a single SM as a unit***
- multiple blocks may run on a single SM simultaneously, up to the *resource limit* of an SM (more on this later)
- GPU keeps SMs occupied with the *right* number of blocks

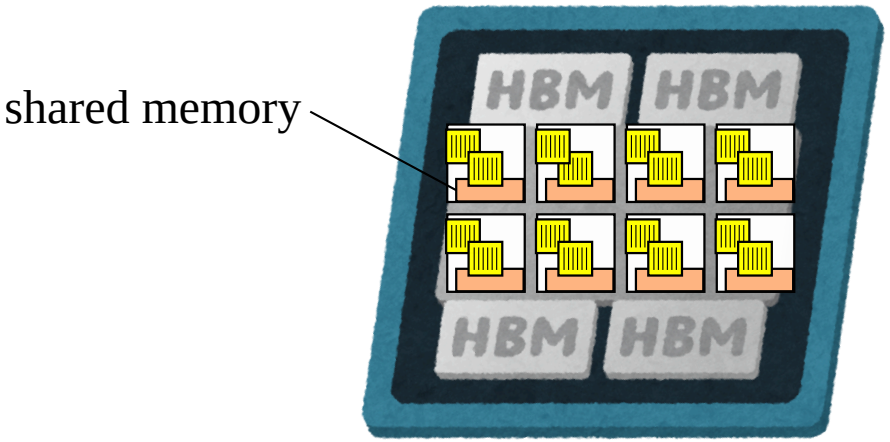


The way thread block boundaries are *semantically* visible



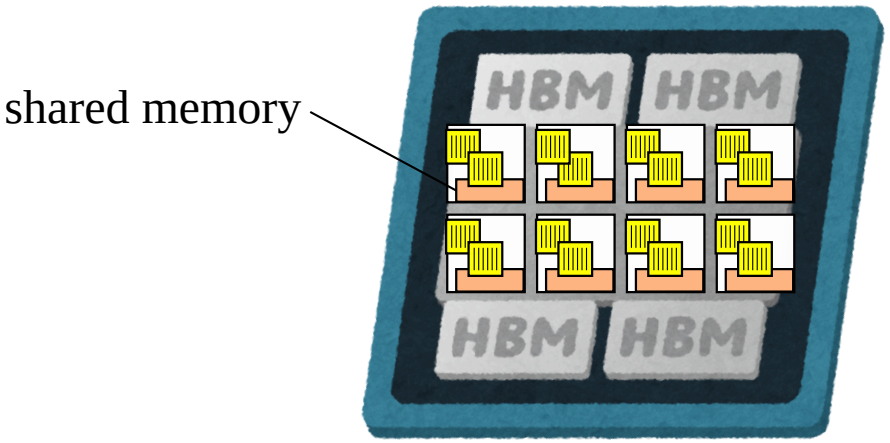
The way thread block boundaries are *semantically* visible

1. ***shared memory*** : a small cache-like memory *only shared within a single thread block*
   - I hate this misnomer



The way thread block boundaries are *semantically* visible

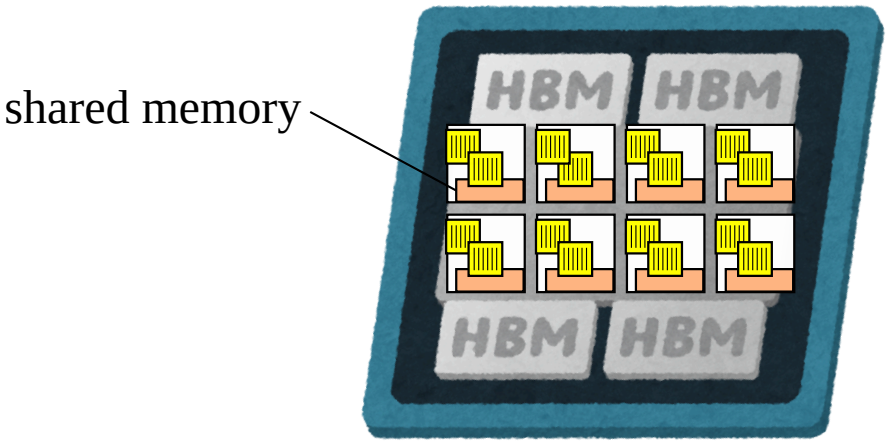
1. ***shared memory*** : a small cache-like memory *only shared within a single thread block*
   - I hate this misnomer
2. **`__syncthreads()`** : a synchronization primitive only usable within a single thread block



The way thread block boundaries are *semantically* visible

1. ***shared memory*** : a small cache-like memory *only shared within a single thread block*
   - I hate this misnomer
2. **`__syncthreads()`** : a synchronization primitive only usable within a single thread block

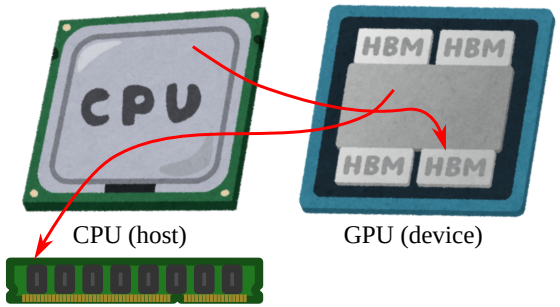
- unless you rely on these primitives, choosing the thread block size is a performance, not a correctness, issue



Data sharing

Data sharing between CPU and GPU

- To cut the long story short,
  - until recently, CPU memory and GPU memory were *separate*
  - recent GPUs support transparent shared memory between CPU and GPU at the hardware level (just as all cores in a CPU do), though they may still be *physically* separate
    - NVIDIA term : NVLink-C2C
    - AMD term : Infinity Fabric
- Purely from correctness standpoint, no need to worry about “is this data on CPU or GPU?”



The following “just works” with CPU-GPU shared memory

```
__global__ void f(int * p) { *p = ... }
int main() {
  int a = ...;
  f<<<nb,bs>>>(&a);
  cudaDeviceSynchronize();  // wait for the kernel to finish
  printf("a = %d\n", a);
}
```

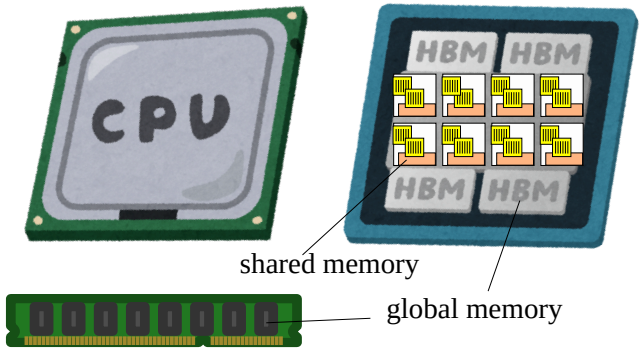
- Equally work for global variables/arrays and heap memory allocated by malloc/new/mmap

Remarks about cudaMalloc/cudaMallocManaged/cudaMemcpy

- `cudaMalloc` : allocates memory accessible *only by the device*
- `cudaMallocManaged` : allocates memory accessible both by device and host
  - existed prior to hardware CPU-GPU cache coherence, but was much slower
- prior to hardware CPU-GPU cache coherence,
  - `cudaMalloc` + explicit `cudaMemcpy` was the norm
  - `cudaMallocManaged` often hurts performance
  - local/global variables cannot be shared
- With hardware CPU-GPU cache coherence,
  - ***ordinarily-allocated memory (global/local/malloc/new/mmap) are accessible both by CPU/GPU***
  - $\Rightarrow$ when correctness is the only concern, no need to use `cudaXXX` at all
  - there *are* implications to performance, which I address separately in later weeks

Data sharing among *threads in the device*

- ***global memory*** ≈ ordinary memory
  - shared by all threads
  - global/local variables/arrays, heap (malloc, new, mmap, cudaMalloc, cudaMallocManaged)
  - i.e., a write by a thread will be visible to all others (sooner or later)
- ***CUDA shared memory***
  - hardware terms: a small on-chip memory as fast as caches, accessible only within each SM
  - software view: memory shared only within a thread block



How to use CUDA shared memory

- declare a shared memory you desire to each thread block

```
f<<<nb,bs,S>>>(...);
```

- from a CUDA program's perspective, it is a memory *only shared within a thread block* and *only active during the thread block's lifetime*
  - the term *shared memory* is a misnomer, IMO; ordinarily-allocated memory *are* shared by all threads
- physically, it is a small cache-like memory faster than global memory
- each SM has a fixed amount of shared memory
- GH200 : 256KB/SM, unified with L1 cache, up to $\approx$ 228KB usable as shared memory

$$S \text{ must be } \leq \text{shared memory per SM}$$

Accessing CUDA shared memory in kernels

- a kernel accesses it by declaring variables or arrays with `__shared__`

```
__shared__ int a[n];
__shared__ char b[m];
```

- if the data size (n or m above) is not a compile-time constant, obtain the starting address of the shared memory by

```
extern __shared__ char whatever[];
```

- it's your responsibility to use the appropriate part of it. e.g.,

```
int * a = (int*)whatever;
char * b = (char *)&a[n];
```

- shared memory is a way to efficiently communicate among threads within a block
- GPUs have hardware-managed caches, so how crucial it is to performance is somewhat changing over time

When data are shared, be careful about race condition

- whether global memory or shared memory, they are subject to race conditions as in CPU

Resolving race condition on CUDA

- atomic accumulations (`atomicAdd` etc. for `#pragma omp atomic`-equivalent)
- no mutual exclusion (no `#pragma omp critical`-equivalent)
- barrier synchronization within and across blocks
- reduction *only within a single thread block*

	OpenMP	CUDA
atomic accumulation	pragma `atomic`	`atomicAdd`
mutual exclusion	pragma `critical` or `omp_lock_t`	
barrier	pragma `barrier`	`__syncthreads()` within a block cooperative thread group
reduction	pragma `reduction`	cooperative thread group only within a block, or build on barrier across blocks

Atomic accumulations

- consider the following (trivial) example

```
int a[n] = { ... };
int s = 0;
  ...
sum<<<nb,bs>>>(a, &s);
cudaDeviceSynchronize();
printf("sum = %d\n", s);
```

- the goal is to guarantee it always prints the sum of all elements in the array
- a race-prone version

```
__global__ void sum(int * a, int * p) {
  int i = <thread id>;
  if (i > 0) *p += a[i];
}
```

Atomic accumulations

- atomic accumulations are supported by the hardware and CUDA API
  - `atomicAdd(p, x)` ≈

```
#pragma omp atomic
  *p += x
```

in OpenMP

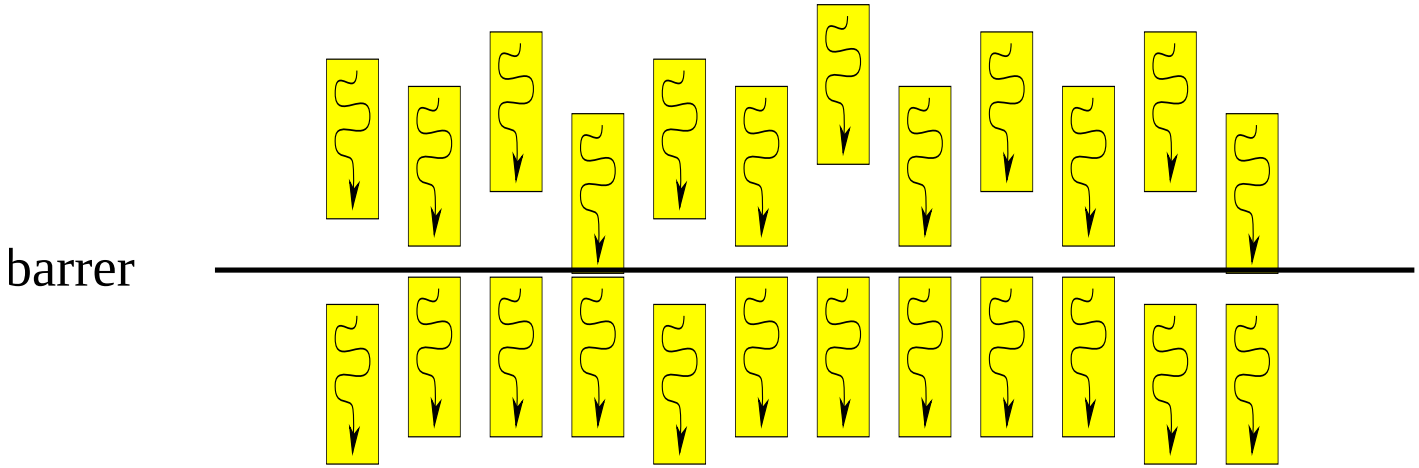
- fix our example

```
__global__ void sum(int * a, int * p) {
  int i = <thread id>;
  if (i > 0) atomicAdd(p, a[i]);
}
```

- this works, but very slow as all threads write to the same variable
- ⇒ build reduction on top of *barrier*

Barrier synchronization

- barrier is a mechanism to ensure “all threads reached a point”
- useful to ensure changes made by a thread are visible to all others
- CUDA used to support barriers only within a single thread block (`__syncthreads()`)
- it now supports barriers for all threads (C. Cooperative Groups), with a caveat (later)



Cooperative groups (1)

- (important) when using the following features, launch a kernel by

```
void * args[] = { a0, a1, ... };
cudaLaunchCooperativeKernel((void *)f, nb, bs, args);
```

  instead of the ordinary

```
f<<<nb,bs>>>(a0, a1, ...);
```

- common setup

```
#include <cooperative_groups.h>
namespace cg = cooperative_groups; // save typing
```

Cooperative groups (2)

- data representing a *group of threads*

```
cg::grid_group g = cg::this_grid();           // all threads
cg::thread_block g = cg::this_thread_block(); // thread block
```

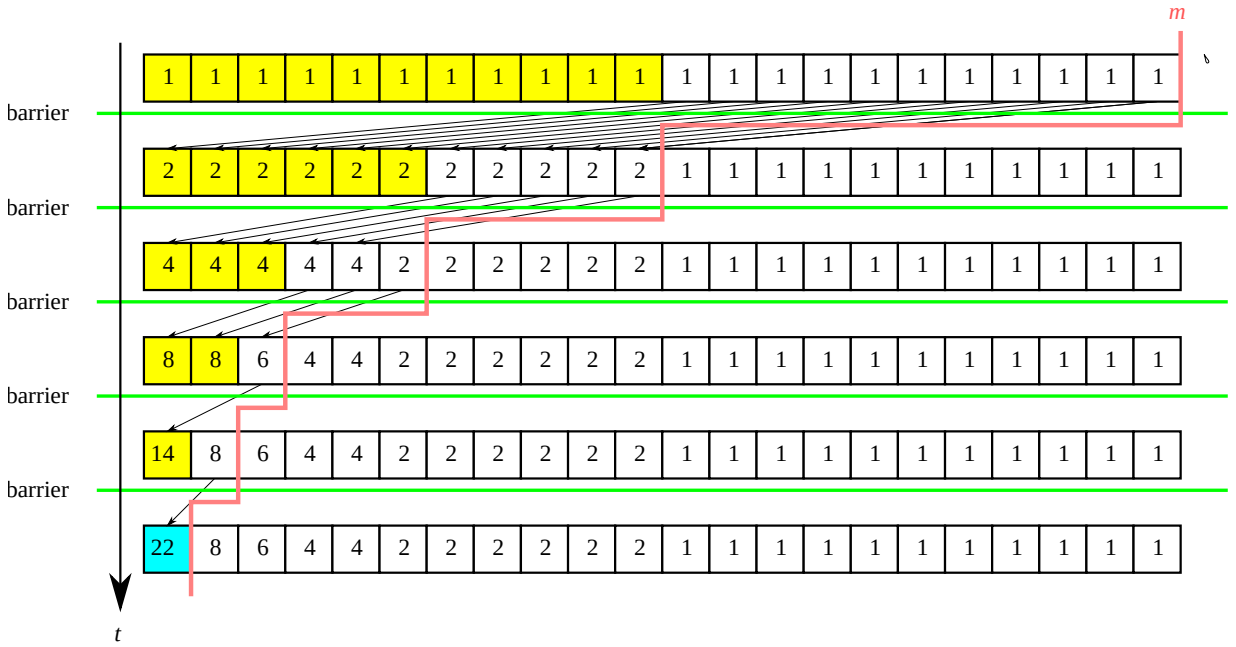
- barrier synchronization

```
g.sync(); // barrier all threads in g participate
```

- BTW, group actually provides a cleaner way to know thread ID and number of threads

```
unsigned long long idx = g.thread_rank(); // my ID in g
unsigned long long nth = g.size();        // num threads in g
```

Building reduction on barrier (basic idea)



- threads in the second half add their values to the first half
  - a[0:n/2] += a[n/2:n]
  - a[0:n/4] += a[n/4:n/2]
  - a[0:n/8] += a[n/8:n/4]
  - …

In code …

```
__global__ void sum(double * c, long n) {
  // return c[0] + .. + c[n-1]
  cg::grid_group g = cg::this_grid();
  ull i = g.thread_rank();
  ull h; // ull: unsigned long long
  for (long m = n; m > 1; m = h) {
    h = (m + 1) / 2;
    if (i + h < m) c[i] += c[i + h];
    g.sync();
  }
}
```

- invariant: “`sum(c[0:m])` is *the* sum”, it repeats halving m
- note: it may not be most efficient; reducing values within a single block first may be better

More on choosing a block size

Recap: parallelizing a for loop

- given a for loop

```
for (i = 0; i < n; i++)
  S(i)
```

1. write a kernel executing a single iteration:

```
__global__ void S_fun(int n) {
  int i         = blockDim.x * blockIdx.x + threadIdx.x;
  if (i < n)
    S(i)
}
```

2. launch n threads (perhaps more than n depending on thread block size)

```
int bs = 64; // more on this later
int nb = (n + bs - 1) / bs; // ensure nb * bs >= n
S_fun<<<nb,bs>>>(n);
```

but why `bs = 64`?

Question

- Assuming the total number of threads (`nb` $\times$ `bs`) is fixed (say, 10,000,000), how to choose the right *block size* (the number of threads dispatched as a unit)?
- Which of the following is good, among countless others?

nb	bs
10,000,000	1
5,000,000	2
…	…
312,500	32
156,250	64
…	…
10,000	1000
5,000	2000
2,500	4000
…	…

Eliminated by simple hardware limit and characteristic

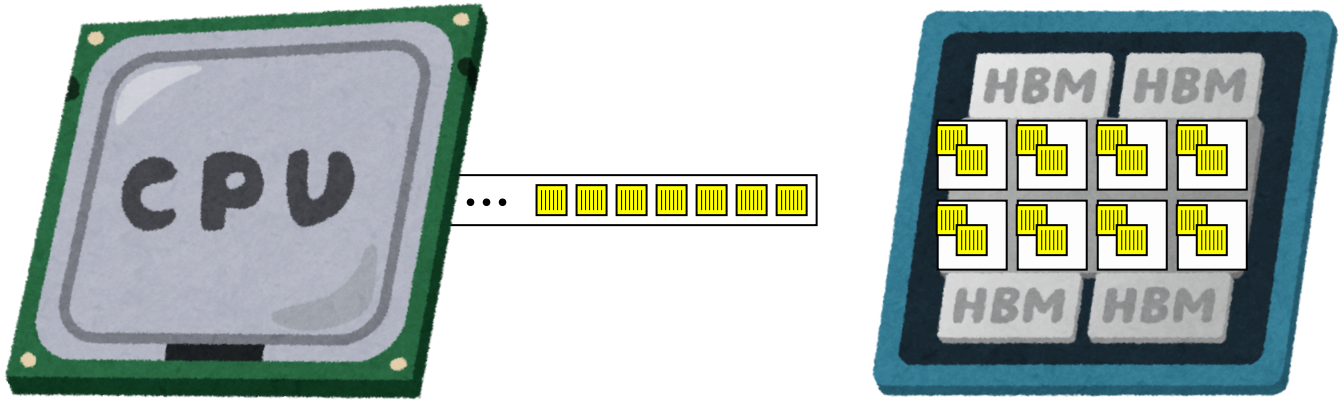
- a block size must be ≤ 1024
- any value not a multiple of 32 is often simply a waste (*warp*)
- $\Rightarrow$ narrowing the choice to [32, 64, 96, 128, …, 1024]

nb	bs
312,500	32
156,250	64
104,166	96
…	…
9,766	1024

Warp

- a *warp* is the unit of *instruction execution* within an SM
- 32 threads in a single warp share an instruction pointer (a warp $\approx$ a CPU thread executing 32-way SIMD instructions)
- at every cycle, an SM selects up to a few (typically two or four) ready warps and executes them
- $\Rightarrow$
  - ***there is rarely a point in making block size $<$ 32 or not a multiple of 32*** (remainder threads consume resources but perform no useful work)
  - you want to make 32 threads branch in the same way (avoid *warp divergence*)

Remember this (execution model)?



- *multiple blocks may run on a single SM simultaneously, up to the* resource limit *of an SM (more on this later)*
- *GPU keeps SMs occupied with the* right *number (**two** in this figure) of blocks*

How GPU determines the number of simultaneously running blocks on an SM

- We call thread blocks actually running on the device ***resident*** blocks
- Let M be the max number of resident blocks per SM for a kernel
- M is determined by:
  - A simple hard-wired limit

$$M \leq \text{a constant specific to hardware (e.g., 32)}$$

  - and two kernels-specific constraints
    - *registers:*

$$\text{register used per thread} \times \text{block size} \times M \leq \text{register capacity of an SM}$$

    - *shared memory:*

$$\text{shared memory used by a block} \times M \leq \text{shared memory capacity of an SM}$$

- Given these constraints, ***the objective is to maximize the number of resident threads***

A reason to prefer smallest values (e.g., 32 or 64)

- For register/shared memory constraint, if we assume the required resource per block $\propto$ block size, there is *no point* in choosing anything other than the minimum possible value
- e.g., for registers, let
  - $R_0 =$ register used per thread
  - $B =$ chosen block size
  - $R =$ register capacity
- then

$$R_0 B M \leq R \rightarrow M = \left\lfloor \frac{R}{R_0 B} \right\rfloor$$

- therefore,

$$\text{the number of resident threads} = B \left\lfloor \frac{R}{R_0 B} \right\rfloor$$

A reason to prefer smallest values (e.g., 32 or 64)

- a similar argument holds for shared memory as long as we assume shared memory per block $\propto$ block size
- altogether, we want to maximize this

$$\text{the number of resident threads} = B \cdot \min\left(\left\lfloor \frac{R}{R_0 B} \right\rfloor, \left\lfloor \frac{S}{S_0 B} \right\rfloor, 32\right)$$

- for the first two terms in the min, choosing the minimum value 32 is always the best,
- except it may make $32B$ too small to fill SM with enough threads
- to cut the long story short, 32 is too small and 64 is good enough
- $\Rightarrow$ 64 is a good choice absence of other reasons

Reasons you may want to choose larger block sizes

- when you want to make sure many threads will run on the same SM, simultaneously
- this happens typically when
  - threads communicate or synchronize extensively
  - threads take advantage of shared memory
  - you want to guarantee only one block is running simultaneously (so you know which threads are running)
- these are algorithm-specific
- 64 is merely a safe default when no reasons exist for larger blocks